

Optimized Local AI Agents for Excel Files: Enabling Accurate Generation and Privacy Protection

Ziwen Wang¹[0009-0000-1724-9353], Chenxiang Xie¹[0009-0009-3733-0154],
Qing Wang¹[0009-0003-0446-577X], and Yong Zhang^{1*}[0000-0001-8803-2055]

BNRist, Tsinghua University, Beijing, China

Abstract. With the rapid development of AI agents, deploying agents in local environments for automated office document generation has emerged as a significant research direction, particularly in socially sensitive contexts where data privacy and trust are critical. This study presents a locally deployable AI agent designed for automated Excel file generation and proposes targeted optimization strategies with a focus on tool integration and prompt engineering. A comprehensive evaluation across diverse task sets demonstrates that the optimizations significantly enhance content accuracy, formula correctness, and structural fidelity. These findings confirm the feasibility of local Excel agents and provide practical insights into designing efficient solutions for automated document generation while ensuring data privacy and security.

Keywords: AI Agent · Spreadsheet Generation · Privacy Protection

1 Introduction

In recent years, Large Language Models (LLMs) have demonstrated remarkable capabilities in text generation and reasoning across various natural language processing (NLP) tasks. Owing to advancements in the Transformer architecture [23], large-scale pre-training, and Reinforcement Learning with Human Feedback (RLHF) [17], state-of-the-art LLMs now possess strong abilities in language understanding and instruction-following.

However, despite the strong capabilities of proprietary models such as GPT-4 [16], Claude-3 [1], and Gemini-2.5 [5] in document generation tasks, their high effectiveness largely relies on cloud-based computational resources and API access. This poses significant challenges for data privacy and access control when handling sensitive information. Furthermore, the high service costs, network latency, and vendor dependencies associated with cloud solutions pose significant challenges to their deployment in industrial environments. From a social computing perspective, these challenges reflect broader concerns about data sovereignty, algorithmic transparency, and equitable access to AI infrastructure [22,11,13].

* Corresponding author. Email: zhangyong05@tsinghua.edu.cn

In sectors such as finance, healthcare, and government administration, where data confidentiality and regulatory compliance are of paramount importance, the ability to deploy AI systems locally and securely is increasingly recognized as a crucial enabler for trustworthy and socially responsible automation. Local deployment enables organizations to process sensitive data within their own infrastructure, maintain full control over information handling, and minimize reliance on external cloud services, supporting both operational efficiency and compliance with privacy regulations.

Meanwhile, studies exploring the capabilities of locally deployed LLMs in document and spreadsheet generation tasks are relatively limited. Although some prior work has attempted to deploy LLMs locally to mitigate privacy and cost risks [24,29], these models often struggle with structured tasks such as Excel spreadsheet generation. These shortcomings significantly limit the usability of local LLMs in real-world business workflows, where reliability, consistency, and efficiency are essential for operational deployment, and hinder their broader adoption in socially sensitive settings such as secure office automation in financial institutions.

To address these challenges, this study proposes and implements an optimization framework for Excel agents (agent’s basic architecture is shown in Figure 1) and systematically evaluates their capabilities in generating accurate and structurally consistent Excel spreadsheets. The experimental results demonstrate that these optimization strategies substantially improve the accuracy and reliability of Excel file generation, as well as the ability to handle complex spreadsheet operations. Furthermore, practical insights are provided into building efficient and robust Excel agents in local deployment environments.

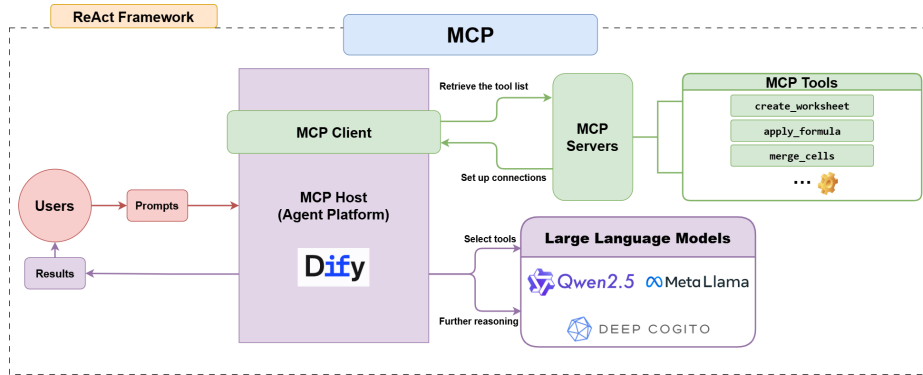


Fig. 1: Basic Architecture of Excel Agents: MCP (Model Context Protocol) provides a standardized architecture that enables AI agents to access external resources, invoke callable functions, and utilize specific prompts for Excel file generation, thereby facilitating seamless communication between the model and its execution environment. This architecture allows the agent to generate structured Excel outputs efficiently within local deployment settings. [2]

2 Related Work

Breakthroughs in large language models (LLMs) have driven their application in structured data generation, particularly in spreadsheet creation and filling tasks [14,8]. To address the conflict between context length and structural fidelity in long-table scenarios, SpreadsheetLLM [7] proposes an efficient encoding framework that explicitly models spreadsheet layouts and cell contexts, and its SheetCompressor module demonstrates strong capabilities in table detection and QA tasks. TableLLaMA [28] leverages the high-quality multi-task table instruction dataset TableInstruct for instruction tuning, significantly enhancing generalization across tasks and domains in various table-related scenarios.

In terms of table structure understanding, TUTA [25] introduces a tree-based Transformer pre-training architecture that models both spatial and hierarchical features of tables via a bi-dimensional coordinate tree and incorporates structured attention and positional embeddings, enabling efficient representations at the token, cell, and table levels. TableFormer [15] significantly improves table structure recognition in table structure understanding tasks.

For formula and computation accuracy in Excel file generation, FLAME [10] is a compact transformer model for Excel formulas, leveraging specific objectives for formula repair, completion, and retrieval. SpreadsheetCoder [3] models formula prediction as a semi-structured context learning problem, leveraging local and cross-column information within worksheets to enhance formula generation accuracy. SpreadNaLa [20] provides a dataset of natural language to spreadsheet formulas for evaluating the capabilities of code generation models.

Regarding the application of tools and multi-stage control in table-focused AI agents, SheetCopilot [12] decomposes user natural language instructions into atomic actions and employs a state machine for multi-stage task planning, enabling LLMs to robustly control spreadsheet software for complex operations. SheetAgent [4] is an autonomous agent for spreadsheet manipulation, composed of three collaborative modules—Planner, Retriever, and Informer—and leverages iterative reasoning and reflection to achieve high-precision spreadsheet operations and reasoning in long-horizon, multi-step tasks without human intervention. Additionally, general reasoning and acting paradigms such as ReAct [27] and self-reflection mechanisms like Reflexion [21] provide strategies for language-based feedback and action loops, effectively enhancing the completion quality of complex tasks without requiring model weight updates.

3 Methodology

3.1 Model Selection

This study selected commonly used large models for local deployment with parameters of 70B and 72B, including llama3.3:70B [9], cogito:70B [6], and qwen2.5:72B [19].

3.2 Design of the Test Case Suite

A test case suite was designed to comprehensively evaluate the basic capabilities of the AI agent. The test case suite consists of both test cases and corresponding prompts. All test cases were based on self-designed Excel files, totaling ten in number. For each test case, tailored prompts were designed to guide the agent in generating the corresponding Excel file.

Three categories of prompt tasks were designed to systematically assess the agent’s capabilities across scenarios of varying complexity: simple tasks, medium tasks, and complex tasks, covering scenarios ranging from basic spreadsheet generation to multi-sheet, complex operations.

Simple tasks feature clear table structures and require only content filling, without formula computation. Medium tasks add Excel formula writing and calculation requirements. Complex tasks involve multi-sheet structures, hierarchical levels, and complex formatting.

This hierarchical task design allows for a comprehensive evaluation of the AI agent’s spreadsheet generation capabilities: (1) simple tasks validate whether the model can correctly generate structured tables; (2) medium tasks assess the model’s ability to understand and write basic formulas; (3) complex tasks evaluate the model’s capabilities in handling cross-sheet structures and complex formatting.

3.3 Evaluation Metrics and Testing Standards

To comprehensively assess the capabilities of the locally deployed AI agent, an automated framework was designed to perform cell-by-cell comparisons between template and generated files, enabling quantitative assessment of content, formulas, and structure. All metrics are normalized to the range $[0, 1]$.

Content Accuracy Content accuracy measures the consistency between the cell values in the generated Excel file and the corresponding values in the template. For all non-empty cells, if the predicted value exactly matches the template value, it is counted as a correct match (score = 1); otherwise, the score is 0. Let N be the total number of compared cells and M the number of exact matches:

$$\text{Content Accuracy} = \frac{M}{N}$$

Content Similarity Considering that minor deviations may occur in generated results due to differences in numeric precision, date formatting, or string casing, this study introduces a string similarity metric to measure the closeness between the generated value and the target value. The similarity is computed using the SequenceMatcher [18] algorithm, where a value closer to 1 indicates higher similarity.

Structure Match Structure match evaluates the consistency between the number and names of worksheets in the generated file and the template. If all worksheets in the template exist in the generated result, the score is 1; otherwise, the score is calculated proportionally based on the number of matching sheets. Let S_r be the set of sheets in the template and S_g in the generated file:

$$\text{Structure Match} = \frac{|S_r \cap S_g|}{|S_r|}$$

Formula Preservation In tasks requiring the use of Excel built-in formulas, the model is expected to generate correct content while maintaining and applying formulas properly. This is verified by loading the template and generated files twice: Using `data_only=False` to check whether formulas are correctly retained in the corresponding cells; Using `data_only=True` to validate whether the computed values derived from those formulas are correct. Let F be the number of formula cells in the template and $F_{\text{preserved}}$ those retained as formulas:

$$\text{Formula Preservation} = \frac{F_{\text{preserved}}}{F}$$

Header Fidelity Table headers are essential for semantic consistency in Excel documents, as errors may lead to data misinterpretation. Header similarity is therefore evaluated only on first-row cells, with a similarity score above 0.9 indicating correct generation.

$$\text{Header Fidelity} = \frac{\text{Matched Headers (Similarity} > 0.9)}{\text{Total Headers}}$$

3.4 Optimization Strategies

In the unoptimized state, the AI agent exhibited limited capability in generating accurate and structurally consistent Excel files, with issues such as missing data and incorrect row or column ordering. This is largely because large language models treat dictionaries as unordered structures internally, which can lead to data being rearranged during output generation and, consequently, a decline in overall output quality.

To address these issues, this study proposes an optimization approach focusing on both tool-level integration and prompt-level structuring. Figure 2 depicts the structure of the optimized prompts proposed in this study.

Tool-Level Optimization At the tool integration level, the Excel generation pipeline was redesigned to support fixed two-dimensional array inputs and outputs, ensuring that the order of rows and columns remains stable throughout the generation process.

Prompt-Level Optimization Prompt engineering was enhanced by incorporating a structural definition of the two-dimensional array, ensuring that outputs adhere to the predefined target structure.

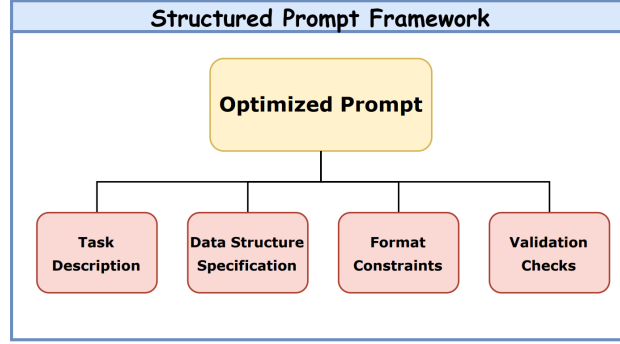


Fig. 2: Structured Prompt Framework: An optimized modular framework that decomposes prompts into task description, data structure specification, format constraints, and validation checks, thereby enhancing overall reliability and consistency of outputs.

3.5 Experimental Procedure

Experimental procedures are as follows:

First Evaluation (Pre-Optimization): Excel files were generated using the basic template without any logical optimization of the agent.

Second Evaluation (Post-Optimization): Excel files were generated using the optimized template, and the outputs were collected.

Result Analysis: Python scripts were employed to automatically compare the generated Excel files from both evaluation rounds with the test cases, and to compute the corresponding evaluation metrics.

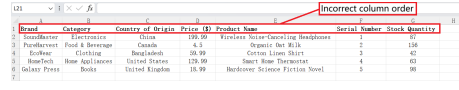
4 Results and Analysis

4.1 Common Error Analysis

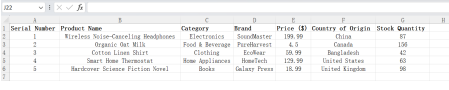
In the unoptimized state, the Excel files generated by the agent exhibit issues in two main aspects: column ordering and formula generation. The latter is largely caused by reference misalignment resulting from incorrect column order. These issues were resolved through optimization.

Column Order Error As shown in Figure 3, errors in the header sequence of the Excel generated by the agent directly lead to disordered columns.

Formula Generation Error As shown in Figure 4, incorrect header sequences disrupt column order, causing cascading errors in formulas, aggregations, and data validations, and ultimately leading to incorrect cell references and range selections.

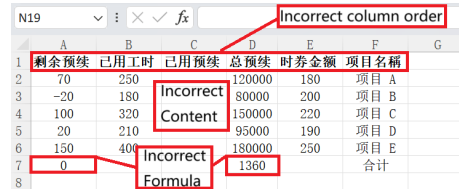


(a) Pre-Optimization

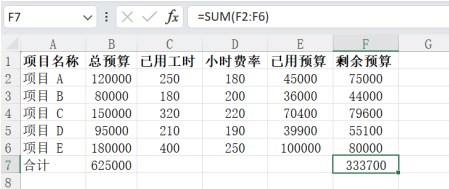


(b) Post-Optimization

Fig. 3: Column Order Error



(a) Pre-Optimization



(b) Post-Optimization

Fig. 4: Formula Generation Error

4.2 Capability Analysis of Excel Generation by Agents under Different Models

The weighted average capability of each model across five evaluation metrics was calculated. Specifically, for the four metrics — Content Accuracy, Content Similarity, Structure Match, and Header Fidelity — each involving ten different prompts, the experimental results for all prompts under the same metric were aggregated and then averaged to reflect the overall capability of the model on that metric. For the Formula Preservation metric, which was tested only under two specific prompts, the two results were summed and averaged accordingly. The resulting 30 data points are summarized in Table 1 and Figure 5.

Overall, after applying the optimization strategies, the Excel agent demonstrated significant and consistent improvements across four key metrics: Content Accuracy, Content Similarity, Formula Preservation, and Header Fidelity. It is worth noting that the Structure Match metric already reflected a relatively strong baseline capability prior to optimization, and this high level of structural generation fidelity was successfully preserved and reinforced afterward.

From the perspective of model comparison, the three tested models show highly comparable improvements across all metrics, with negligible differences. This suggests that system-level enhancements, including prompt engineering and

tool optimization, play a role comparable to the intrinsic capabilities of the underlying models.

4.3 Capability Analysis of Excel Generation by Agents under Different Prompts

Section 4.2 indicates that, under the experimental settings of this study, differences among models have a relatively limited impact on the final generated results. Therefore, when analyzing the capabilities of the Excel agent across different prompts, the three sets of experimental results can be treated as three independent trials of the same model under identical prompt conditions. Accordingly, the arithmetic mean of the results for the three models was calculated for each prompt. The resulting 84 data points are summarized in Table 2 and Figure 6.

The data show that, after applying the optimization strategies, all prompts lead to a significant improvement in the quality of the Excel files generated by the agent. For simple tasks (Prompt 1, Prompt 4, Prompt 7, Prompt 8, Prompt 9, and Prompt 10) and medium tasks (Prompt 2 and Prompt 6), all evaluation metrics for the agent reached values above 0.9. In contrast, for complex tasks (Prompt 3 and Prompt 5), which involve operations such as cross-sheet references and merged cells, the magnitude of improvement is comparatively smaller, reflecting the higher inherent difficulty of these tasks. The study further observed that when the agent executes merged cell operations and content population simultaneously, the header fidelity metric consistently remains at zero. In contrast, decomposing the task into sequential steps—merging cells followed by content insertion—substantially improves the accuracy of both header content and header structure. These findings suggest that, for complex Excel generation tasks, employing a stepwise prompt design strategy can markedly enhance the agent’s accuracy in preserving both content integrity and structural fidelity.

All scripts, datasets, and the experimental environment are available at the external link [26].

Table 1: Capability Comparison of Agents on Excel Generation under Different Models

Model	Content Accuracy	Content Similarity	Structure Match	Formula Preservation	Header Fidelity
llama3.3:70b	0.21	0.30	1.00	0.10	0.22
qwen2.5:72b	0.24	0.33	0.95	0.10	0.25
cogito:70b	0.22	0.32	0.95	0.10	0.26
llama3.3:70b-opt	0.99	0.99	1.00	1.00	0.90
qwen2.5:72b-opt	0.99	0.99	0.95	1.00	0.90
cogito:70b-opt	0.98	0.99	1.00	1.00	0.90

Table 2: Capability Comparison of Agents on Excel Generation under Different Prompts

Prompt	Content Accuracy	Content Similarity	Structure Match	Formula Preservation	Header Fidelity
1	0.98	1.00	1.00	null	0.95
2	0.25	0.25	1.00	0.20	0.25
3	0.26	0.34	0.67	null	0.26
4	0.00	0.18	1.00	null	0.00
5	0.00	0.00	1.00	null	0.00
6	0.06	0.21	1.00	0.00	0.11
7	0.17	0.43	1.00	null	0.29
8	0.14	0.23	1.00	null	0.14
9	0.28	0.38	1.00	null	0.33
10	0.08	0.12	1.00	null	0.08
1-opt	1.00	1.00	1.00	null	1.00
2-opt	1.00	1.00	1.00	1.00	1.00
3-opt	1.00	1.00	0.83	null	1.00
4-opt	1.00	1.00	1.00	null	1.00
5-opt	0.95	0.95	1.00	null	0.00
6-opt	1.00	1.00	1.00	1.00	1.00
7-opt	1.00	1.00	1.00	null	1.00
8-opt	1.00	1.00	1.00	null	1.00
9-opt	0.99	1.00	1.00	null	1.00
10-opt	0.90	0.96	1.00	null	1.00

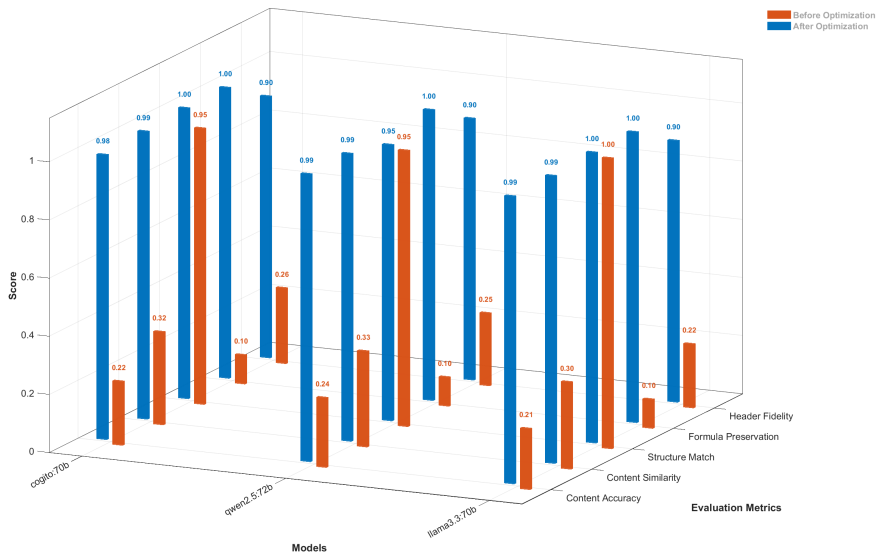


Fig. 5: Capability Analysis of Excel Generation Across Different Models

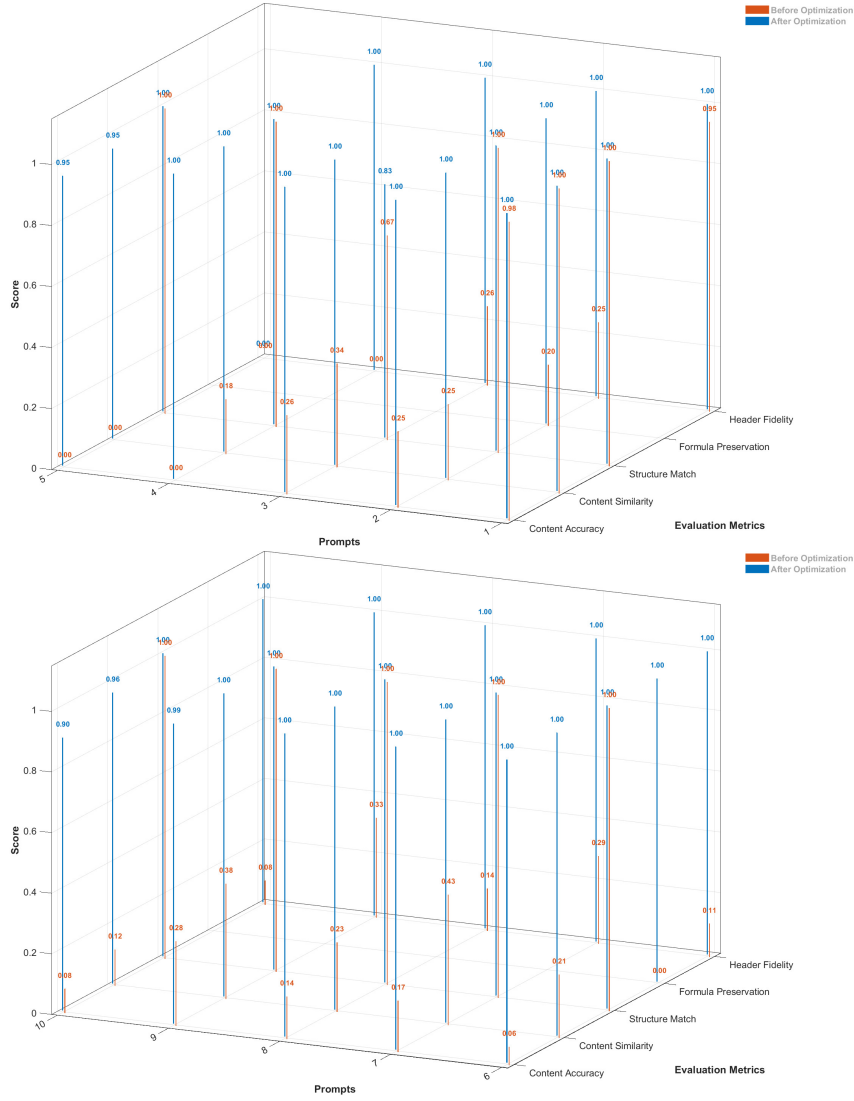


Fig. 6: Capability Analysis of Excel Generation Across Different Prompts

5 Conclusion and Application in Social Computing

This study conducts a systematic investigation into the construction and capability optimization of AI agents for Excel tasks under resource-constrained hardware environments. The main contributions of this work are threefold: (1) A lightweight Excel agent architecture is proposed and implemented, capable of efficient operation under limited computational resources, such as consumer-grade GPUs, providing a feasible solution for low-cost intelligent office automation,

while mitigating risks to data privacy and ensuring secure local processing. (2) A comprehensive Excel agent test suite covering diverse scenarios and requirements is constructed, and the capabilities of three selected open-source Large Language Model-based agents are systematically evaluated across key metrics. (3) System-level optimization strategies for the Excel agent are proposed and validated. Experimental results demonstrate that these measures achieve significant improvements across multiple core metrics, with consistent and stable gains.

Beyond technical performance, the potential applications of this locally deployed AI agent extend to domains that demand stringent data protection and confidentiality. In industries such as banking, insurance, and securities, on-device processing allows organizations to manage sensitive information securely, minimize dependence on cloud services, and comply with regulatory requirements. These cases demonstrate that locally deployed AI agents not only enhance the efficiency of office automation but also promote responsible and trustworthy AI practices.

References

1. The claude 3 model family: Opus, sonnet, haiku. <https://api.semanticscholar.org/CorpusID:268232499>
2. Anthropic: Model context protocol. <https://www.anthropic.com/news/model-context-protocol> (2024), accessed: 2025-10-19
3. Chen, X., et al.: Spreadsheetcoder: Formula prediction from semi-structured context (2021), <https://arxiv.org/abs/2106.15339>
4. Chen, Y., et al.: Sheetagnt: Towards a generalist agent for spreadsheet reasoning and manipulation via large language models (2024), <https://arxiv.org/abs/2403.03636>
5. Comanici, G., et al.: Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities (2025), <https://arxiv.org/abs/2507.06261>
6. Deep Cogito: Cogito v1 preview. <https://www.deepcogito.com/research/cogito-v1-preview> (2025), accessed: 2025-08-29
7. Dong, H., et al.: Spreadsheetllm: Encoding spreadsheets for large language models (2024), <https://arxiv.org/abs/2407.09025>
8. Fang, X., et al.: Large language models (llms) on tabular data: Prediction, generation, and understanding – a survey (2024), <https://arxiv.org/abs/2402.17944>
9. Grattafiori, A., et al.: The llama 3 herd of models (2024), <https://arxiv.org/abs/2407.21783>
10. Joshi, H., et al.: Flame: A small language model for spreadsheet formulas (2023), <https://arxiv.org/abs/2301.13779>
11. Kashima, H., Oyama, S., Arai, H., Mori, J.: Trustworthy human computation: A survey (2022), <https://arxiv.org/abs/2210.12324>
12. Li, H., et al.: Sheetcopilot: Bringing software productivity to the next level through large language models (2023), <https://arxiv.org/abs/2305.19308>
13. Li, T., Das, S., Lee, H.P., Wang, D., Yao, B., Zhang, Z.: Human-centered privacy research in the age of large language models (2024), <https://arxiv.org/abs/2402.01994>

14. Lu, W., et al.: Large language model for table processing: a survey. *Frontiers of Computer Science* **19**(2) (Jan 2025). <https://doi.org/10.1007/s11704-024-40763-6>, <http://dx.doi.org/10.1007/s11704-024-40763-6>
15. Nassar, A., et al.: Tableformer: Table structure understanding with transformers (2022), <https://arxiv.org/abs/2203.01017>
16. OpenAI, et al.: Gpt-4 technical report (2023), <https://arxiv.org/abs/2303.08774>
17. Ouyang, L., et al.: Training language models to follow instructions with human feedback (2022), <https://arxiv.org/abs/2203.02155>
18. Python Software Foundation: difflib — helpers for computing deltas. <https://docs.python.org/3/library/difflib.html> (2025), accessed: 2025-08-29
19. Qwen, et al.: Qwen2.5 technical report (2024), <https://arxiv.org/abs/2412.15115>
20. Schuster, S., et al.: SpreadNaLa: A naturalistic code generation evaluation dataset of spreadsheet formulas. In: Calzolari, N., Kan, M.Y., Hoste, V., Lenci, A., Sakti, S., Xue, N. (eds.) *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. pp. 15216–15225. ELRA and ICCL, Torino, Italia (May 2024), <https://aclanthology.org/2024.lrec-main.1323/>
21. Shinn, N., et al.: Reflexion: Language agents with verbal reinforcement learning (2023), <https://arxiv.org/abs/2303.11366>
22. Shneiderman, B.: Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human-Computer Interaction* **36**(6), 495–504 (2020). <https://doi.org/10.1080/10447318.2020.1741118>, <https://doi.org/10.1080/10447318.2020.1741118>
23. Vaswani, A., et al.: Attention is all you need (2017), <https://arxiv.org/abs/1706.03762>
24. Wang, X., et al.: Empowering edge intelligence: A comprehensive survey on on-device ai models. *ACM Computing Surveys* **57**(9), 1–39 (Apr 2025). <https://doi.org/10.1145/3724420>, <http://dx.doi.org/10.1145/3724420>
25. Wang, Z., et al.: Tuta: Tree-based transformers for generally structured table pre-training. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. p. 1780–1790. KDD '21, ACM (Aug 2021). <https://doi.org/10.1145/3447548.3467434>, <http://dx.doi.org/10.1145/3447548.3467434>
26. Wang, Z., Xie, C.: Materials (2025), https://1drv.ms/f/c/0b70ed28452aa8bc/EuYW_9g4LB1IodnJqFYqv1kBNhXAArFY1qh95Hgq3M47Q?e=04ofi6, accessed: 2025-08-29
27. Yao, S., et al.: React: Synergizing reasoning and acting in language models (2022), <https://arxiv.org/abs/2210.03629>
28. Zhang, T., et al.: Tablellama: Towards open large generalist models for tables (2023), <https://arxiv.org/abs/2311.09206>
29. Zheng, Y., et al.: A review on edge large language models: Design, execution, and applications (2024), <https://arxiv.org/abs/2410.11845>